VLSI AUTOPILOT

A Multi-Agent Intelligent EDA Flow Automation and PPA Optimization Framework

Project Report

Submitted by: Saanyu Varma

Department: Electronics and Communication Engineering (ECE)

Institution: Bhoj Reddy Engineering College for Women

Domain: VLSI Design, EDA Automation, RTL Design and AI Agents

Generated: 13 October 2026

Table of Contents

1. Abstract

2. Introduction and Problem Statement

3. Objectives

4. Software Requirements

5. System Design / Flowchart

6. Implementation (Linux Commands and Shell Scripts)

7. Output Screenshots and Results

8. Conclusion

9. Future Enhancements

10. References

1. Abstract

VLSI AutoPilot is a modular starter framework for automating selected digital IC design tasks. It combines Verilog RTL, functional simulation, logic synthesis, deterministic software agents, report collection, and experiment tracking. The initial design is a four-bit ALU supporting addition, subtraction, bitwise AND, and bitwise XOR. Icarus Verilog is used for simulation and Yosys for synthesis when installed successfully. Timing, power, and foundry-specific physical area are not claimed unless suitable tools, libraries, and constraints are configured.

Keywords: VLSI, EDA Automation, Verilog, RTL, Multi-Agent Framework, Synthesis, PPA.

2. Introduction and Problem Statement

VLSI design includes RTL development, functional verification, synthesis, timing analysis, power estimation, and physical implementation. Engineers must coordinate tools, inspect logs, debug failures, and compare alternatives. VLSI AutoPilot automates a small reproducible simulation-and-synthesis flow and records available evidence.

Problem statement: repetitive tool execution and scattered logs make results difficult to track. The prototype centralizes execution status, synthesis outputs, and experiment records without claiming to replace a complete industrial RTL-to-GDSII flow.

3. Objectives

• Automate simulation and synthesis for a small Verilog design.

• Verify the ALU using 1024 input/operation combinations.

• Capture logs and preserve the synthesized netlist.

• Coordinate deterministic agents using Python.

• Record run metadata and available generic synthesis statistics.

• Clearly mark unmeasured timing, power, and physical area.

Agent	Responsibility
RTL Analysis	Basic RTL structural checks
Verification	Reads actual simulation log

Agent	Responsibility
Synthesis	Checks synthesis log and netlist
Area	Reports generic statistics with caveats
Timing	Not configured in starter flow
Power	Not configured in starter flow
Debug	Flags failures for investigation
PPA Optimization	Baseline only until a candidate is tested
Experiment Manager	Stores agent outcomes

4. Software Requirements

Tool	Purpose
Python 3	Orchestration and reporting
Icarus Verilog	RTL simulation
Yosys	Logic synthesis
Bash	Automation script
ReportLab	PDF generation
OpenSTA/OpenROAD/power tools	Optional advanced stages; not configured

A browser-based Linux notebook can be used from Android. Its storage is temporary; download the generated archive before ending the session.

5. System Design / Flowchart

The design-input layer receives RTL and testbench files. The orchestrator runs the analysis and EDA stages. The analysis layer reads actual logs, while the reporting layer saves metrics and creates the report.

```
START\n  |\nLoad RTL and testbench\n  |\nBasic RTL checks\n  |\nRun simulation\n  |\nPASS? -- no --> Record failure and
```

PPA comparison must use equivalent tools, constraints, libraries, and measurement definitions. Generic cell counts are not foundry-specific physical area.

6. Implementation (Linux Commands and Shell Scripts)

The notebook creates rtl/alu4.v, testbench/alu4_tb.v, scripts/run_flow.sh, Python agent modules, configs/config.json, logs, reports, and experiments.

```
bash scripts/run_flow.sh\npython main.py\npython generate_report.py
```

The ALU has four operations selected by a two-bit opcode. Since the result is four bits wide, addition and subtraction wrap modulo 16. The self-checking testbench loops over all 16 values of A, all 16 values of B, and all four operations, totalling 1024 cases.

The orchestrator calls deterministic modules for RTL checks, verification, synthesis, timing, area, power, debugging, optimization status, and experiment tracking. It does not claim that an LLM is integrated.

7. Output Screenshots and Results

Metric	Recorded result
Simulation	PASS
Synthesis	PASS
Flow exit code	0

Metric	Recorded result
Generic Yosys cell count	Not parsed; inspect synthesis log
Physical area	NOT MEASURED
Timing	NOT MEASURED
Power	NOT MEASURED

Use genuine screenshots of the project structure, tool versions, simulation output, Yosys statistics, generated netlist, agent output, experiment record, and PDF. Do not substitute mock terminal images.

Interpretation: simulation is PASS; synthesis is PASS. Only actual run logs are evidence. Timing, power, and physical area remain not measured in this starter flow.

8. Conclusion

The prototype demonstrates a reproducible EDA automation framework using a self-checking Verilog testbench, Icarus Verilog, Yosys, Bash, Python agents, and experiment records. Successful simulation and synthesis do not prove physical timing closure or power optimization.

9. Future Enhancements

• Integrate OpenSTA with valid constraints and a compatible cell library.

• Add physical design through a supported OpenROAD/OpenLane flow.

• Add power estimation with appropriate libraries and switching activity.

• Run a second RTL candidate and compare it fairly against the baseline.

• Improve report parsing and preserve Git/configuration hashes.

• Add a dashboard and optional LLM explanations with human review.

10. References

Icarus Verilog: https://steveicarus.github.io/iverilog/

Yosys documentation: https://yosyshq.readthedocs.io/

Python: https://docs.python.org/3/

ReportLab: https://www.reportlab.com/documentation/

EDA Playground: https://eda-playground.readthedocs.io/

Report note: results are loaded from the local JSON created by this notebook. Rerun the flow and regenerate this PDF after new experiments.